Advanced Database Systems Portfolio Documentation

MIT 8103 | 2026/2027 First Semester Continuous Assessment

STUDENT	Timothy Okooboh	STUDENT ID	301915902
CASE STUDY	AsterMart Retail & Fulfilment Ltd.	DATABASES	PostgreSQL 16 + MongoDB 7

Repository: github.com/timothyokooboh/advanced-database-portfolio

Organisation and technical approach

AsterMart is a simulated Nigerian multi-branch retailer that includes online ordering, branch pickup, delivery, electronic payment, and stock management. PostgreSQL is the transactional system of record. MongoDB supports read-heavy product catalog and event-timeline patterns. Docker Compose makes the portfolio reproducible in a local environment.

1. Database design and modelling

The design defines 10 normalized relations for branches, categories, products, inventory, customers, addresses, orders, order items, payments and fulfillment. The ER model captures cardinalities and keys. PostgreSQL constraints ensure natural identifiers, ownership, controlled statuses, positive values and delivery-address rules. The schema is in 3NF and order-line prices are deliberate historical snapshots.

2. Query processing and optimisation

The deterministic dataset includes 20,000 customers, 60,000 orders, 180,000 order items and 60,000 payments. Five business queries were run with EXPLAIN (ANALYZE, BUFFERS) before and after workload-specific indexes. The biggest measured improvement was a 60,000-row branch-workload scan going from 4.017 ms to a 0.034 ms index-only scan. The repository holds plans, results, buffer counts, index sizes, and trade-offs.

EVIDENCE	BASELINE	OPTIMISED	OBSERVATION
Branch workload	4.017 ms	0.034 ms	Index-only range scan
Product sales	8.029 ms	1.118 ms	Avoided 180,000-line scan
Payment window	3.205 ms	0.198 ms	Partial successful-payment index

Practical implementation and advanced models

3. Transactions and concurrency

The place_order function locks inventory rows in deterministic product order, verifies stock, inserts header and lines and deducts inventory atomically. In a two session test, both sessions needed four out of five available units. Session A committed, Session B waited and saw the new balance of one, and rolled back with an insufficient-stock error. A separate rollback test returned an inventory value of 141 to 143.

4. NoSQL and advanced data models

MongoDB: 500 product catalog documents with flexible category attributes and embedded branch availability and 5,000 append-only order events. Strict JSON Schema validators will fail malformed documents. Indexes support SKU identity, category/price browsing, tags, branch availability, text search, and ordered timelines. Explain run used idx_category_price, scanned 32 keys, 32 docs for 32 results.

5. Distributed and cloud database exercise

Two PostgreSQL containers implement logical replication from an operational publisher to a reporting subscriber. Initial data and a subsequent branch-summary change were replicated; publisher and subscriber values matched and pg_stat_subscription showed active receive/apply positions. A read-only reporting account could query the subscriber but its insert was rejected. The analysis distinguishes asynchronous reporting replication from automatic high-availability failover and proposes multi-zone deployment, monitored lag, backups, and branch-oriented distribution only when scale justifies it.

ORDER / INVENTORY WRITES	LOGICAL REPLICATION	REPORTING READS
PostgreSQL publisher port 5441	Publication + WAL changes asynchronous	PostgreSQL subscriber port 5442

Verification highlights

- PostgreSQL and MongoDB services run locally through Docker Compose.
- Automated tests validated row counts, financial totals, schema rules, indexes, rollback postconditions, and NoSQL document diversity.
- Evidence files preserve query results, plans, concurrency output, validation output, and replication status.
- The repository uses one case study consistently and preserves reproducible implementation evidence.

Repository evidence map

PORTFOLIO	PRIMARY EVIDENCE
1 - Design	ERD, DDL, keys, constraints, normalisation
2 - Optimisation	SQL results, before/after plans, index analysis
3 - Transactions	Commit/rollback output, competing-session lock test
4 - NoSQL	MongoDB model, queries, explain and validation
5 - Distributed	Publisher/subscriber output and monitoring state

Reflection, declaration and references

Individual technical reflection

This portfolio showed me that advanced database design is not only about creating tables. Constraints and normalization establish correctness, but workload evidence determines useful indexes, and transaction boundaries determine whether a correct schema remains correct under concurrency. The first seed run failed because a temporary discounted order violated the total constraint; executing the scripts exposed this early. The scarce-stock test made row locks and isolation concrete because the second session waited and rechecked committed stock instead of overselling.

I also learned to choose models by access pattern. MongoDB simplified variable product attributes and event payloads, but duplicating availability creates a consistency boundary, so PostgreSQL must remain authoritative during checkout. Logical replication improved reporting isolation and read availability, yet it did not remove the need for backups, schema coordination, lag monitoring, or planned failover. My main lesson is that design claims should be supported by repeatable scripts, measured plans, negative tests, and documented limitations.

AI use declaration

OpenAI Codex assisted with requirements organisation, drafts of code/documentation, testing ideas, and formatting. I executed all scripts locally and inspected database states. I consulted official documentation for technical claims. I remain responsible for reviewing, understanding, explaining, and submitting the work. The full declaration is included in the repository.

References

1. PostgreSQL Global Development Group, PostgreSQL 16 Documentation, https://www.postgresql.org/docs/16/.
2. PostgreSQL Global Development Group, Logical Replication, https://www.postgresql.org/docs/16/logical-replication.html.
3. MongoDB, Inc., Data Modeling, https://www.mongodb.com/docs/manual/data-modeling/.
4. MongoDB, Inc., Schema Validation, https://www.mongodb.com/docs/manual/core/schema-validation/.